

2. Header Linked List

–Creation of Header List .

Program:

```
typedef struct Node
{
    int data;
    struct Node *next;
} Node;

void createHeaderList()
{
    // Allocate memory for the Header Node
    head = (Node *)malloc(sizeof(Node));

    if (head == nullptr) {
        cout << "Error: Memory allocation for Header
failed." << endl;
        exit(1);
    }

    // internal counter for position validation (metadata)
    head->data = 0;
    head->next = nullptr;
}

int main()
{
    createHeaderList();
}
```

Program Analysis

1. header->data (The Value)

- **Can it be any number? Yes.** Since `data` is defined as an `int`, you can set it to `-1`, `0`, `9999`, or any other integer.
- **Can it be a string or decimal? No.** It must match the type defined in our `struct`. If our `struct` says `int data;` we cannot store `"Hello"` or `3.14` inside it.
- **Does the value matter?**
 - **In a standard Header List:** The value is usually ignored (garbage). Developers often use `-1` or `0` just to indicate "this node is a dummy."
 - **In the Optimized version (my previous reply):** We deliberately used it to store the **Count** of items. In that specific logic, you *must* initialize it to `0`.

2. header->next (The Pointer) — CRITICAL

- **Can it be "anything"? Absolutely NOT.** This is the most dangerous part of pointers.
- **What MUST it be?** When we create the header node (and the list is empty), `header->next` **MUST be set to `NULL`** (or `nullptr` in modern C++).
- **Why?**
 - `NULL` acts as the "Stop Sign."
 - When we write functions like `display()` or `insertAtEnd()`, your `while` loop looks for `NULL` to know when to stop:

``while (temp → next != NULL) { ... }``

- If we leave `header->next` uninitialized or set it to "anything" (a random memory address), it becomes a **Dangling Pointer**. Hence our program will try to read memory it doesn't own, resulting in a **Segmentation Fault (Crash)**.

Hence , we have:

```
head = (Node *)malloc(sizeof(Node));
```

→ *The size of Header Node will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

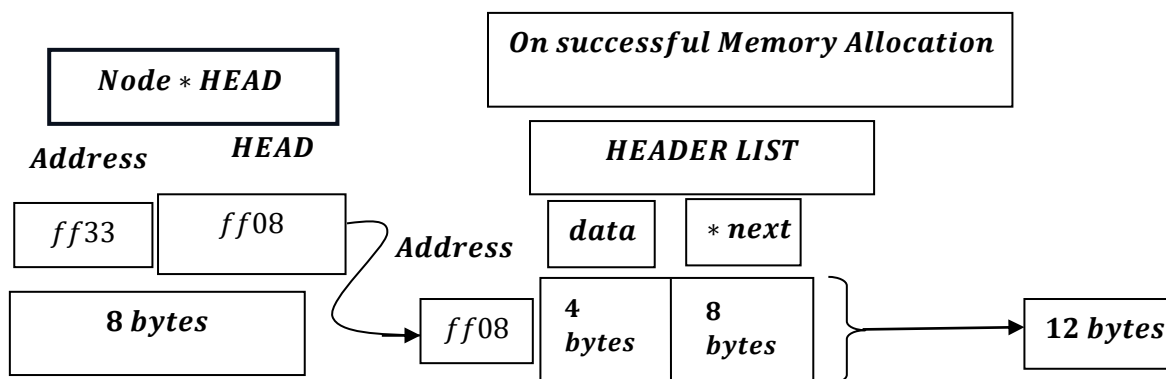
→ *The size of a pointer Header Node is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next) .*



```

void createHeaderList()
{
    // Allocate memory for the Header Node
    head = (Node *)malloc(sizeof(Node));

    if (head == nullptr) {
        cout << "Error: Memory allocation for Header
failed." << endl;
        exit(1);
    }

    // internal counter for position validation (metadata)
    head->data = 0;
    head->next = nullptr;
}

```

Now, head = ff08, hence → Memory Allocation is Successful !

if head = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

And then exit(1) means abnormal termination of the program.

OR ,

Instead of exit(1) → we can use a macro : EXIT_FAILURE .

```

if (head == nullptr) {
    cout << "Error: Memory allocation for Header
failed." << endl;
    //exit(1);
    EXIT_FAILURE;
}

```

Also,

Instead of `exit(0)` → we can use a macro : `EXIT_SUCCESS`.

```
head->data = 0;  
head->next = nullptr;
```

Hence ,on successful memory allocation,we get:

